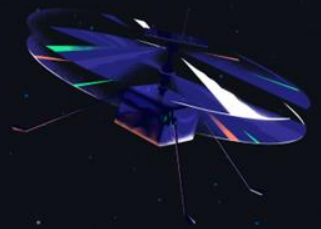
GitHub Copilot for DevOps



AGENDA

DevOps

- Capabilities & Features
- Common Problems
- Hands-on Lab
- Q & A

Outcome of this training

At the end of training, you will achieve the following results...

- ✓ Generate Dockerfiles, Terraform, Bicep, and Azure DevOps pipeline YAML using Copilot chat
- ✓ Analyse log files and write KQL monitoring queries with Copilot assistance
- ✓ Apply the mercury-devops agent skill to auto-load pipeline and scripting standards into every Copilot session

Workshop

Capabilities and Features

Docker

Quickly generate a Dockerfile using GitHub Copilot

Dockerfile

```dockerfile
# a dockerfile based on the official node image to build and run the app

# use the official node image
FROM node:latest

# set the working directory
WORKDIR /usr/src/app

# copy the package.json and package-lock.json files
COPY package*.json ./

# install the dependencies
RUN npm install

# copy the rest of the files
COPY . .

# expose the port
EXPOSE 3000

# run the app
CMD ["npm", "start"]
```

Docker

Prompts to demonstrate:

```
# Basic: generate a Dockerfile for a .NET 8 API
# Prompt: "Create a production-ready Dockerfile for a
.NET 8 Web API with multi-stage build, run as non-
root user, and health check endpoint"

# Security context prompt:
"Review this Dockerfile for security vulnerabilities
and suggest least-privilege improvements"

# Compose prompt:
"Generate a docker-compose.yml for this .NET API with
SQL Server and a Redis cache, using environment
variables for secrets"
```

Infrastructure as Code

Ask Copilot to generate generate IaC templates

terraform > main.tf > resource "azurerm_app_service_plan" "appserviceplan" > sku

```
# Azure App Service linux hosted Node.js application in australia east

resource "azurerm_app_service_plan" "appserviceplan" {
  name                = "appserviceplan"
  location            = "australiaeast"
  resource_group_name = azurerm_resource_group.rg.name
  kind                = "Linux"
  reserved            = true

  sku {
    tier = "Standard"
    size = "S1"
  }
}

resource "azurerm_app_service" "appservice" {
  name                = "appservice"
  location            = "australiaeast"
  resource_group_name = azurerm_resource_group.rg.name
  app_service_plan_id = azurerm_app_service_plan.appserviceplan.id

  site_config {
    linux_fx_version = "DOCKER|node:12.16.1-alpine"
  }
}
```

Infrastructure as Code

Terraform — Mercury naming convention prompt:

```
# Prompt: "Generate a Terraform module for an Azure
App Service Plan following these naming conventions:
# - Resource group: rg-{app}-{env}-{region}
# - App Service Plan: asp-{app}-{env}-{region}
# Use azurerm provider ~> 3.0, B1 SKU, Australia
East"
```

CI/CD Pipelines

Copilot can help to generate your **CI/CD pipelines** too

```yaml
# build and deploy an azure function
name: Build and Deploy Azure Function

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:

  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Setup .NET Core
        uses: actions/setup-dotnet@v1
        with:
          dotnet-version: 3.1.101
      - name: Build with dotnet
        run: dotnet build --configuration Release
      - name: Test with dotnet
        run: dotnet test --no-build --verbosity normal
      - name: Publish with dotnet
        run: dotnet publish --configuration Release --output ./bin/Release/netcoreapp
      - name: Deploy to Azure Function App
        uses: azure/webapps-deploy@v2
        with:
          app-name: 'my'
          slot-name: 'production'
```

CI/CD Pipelines

Azure DevOps YAML prompt:

```
# Prompt: "Generate an Azure DevOps pipeline
YAML that:
# - Builds a .NET 8 solution
# - Runs xUnit tests with code coverage
# - Publishes coverage to Azure DevOps Test
Plans
# - Deploys to Azure App Service in Australia
East
# - Uses a variable group 'mercury-prod-secrets'
for connection strings"
```

Write Scripts

Helps with **script writing** and reading that cryptic bash syntax.

@workspace /explain

GitHub Copilot used @workspace

Used 1 reference

The selected code is a single line of a Bash shell script. It uses the `echo` command to print a message to the terminal. The message includes the value of a special variable, `PWD`, which stands for "Present Working Directory". This variable always holds the full path of the current directory.

The `${PWD%/*}` expression is a form of parameter expansion in Bash. The `%` operator trims the shortest match from the end of the string. In this case, it trims everything from the last slash (`/`) to the end of the string. This effectively removes the current directory from the `PWD` path, leaving only the parent directory.

So, the entire line `echo "My parent folder is: ${PWD%/*}"` prints a message that states the path of the parent directory of the current working directory. For example, if the current directory is `/home/user/documents`, the script will print `My parent folder is: /home/user`.

/explain

Copilot generated code may be incorrect

```
3  echo "My parent folder is: ${PWD%/*}"
```

Write Scripts

PowerShell — Mercury deployment pattern:

```
# Prompt: "Write a PowerShell 7 script that:
# - Deploys an Azure App Service from a ZIP package
# - Validates the deployment health endpoint returns 200
# - Rolls back to the previous deployment slot if health check fails
# - Logs all steps to Azure Log Analytics workspace 'mercury-logs'"
```

Bash — pipeline utility:

```
# Prompt: "Write a bash script to:
# - Check all Azure App Services in a subscription for SSL certificates expiring in <
30 days
# - Send a Teams webhook notification with the list
# - Exit 0 if none found, exit 1 if any found (for use in scheduled pipelines)"
```

Alerts and Monitoring

Quickly generate IaC **alert and monitoring** configuration

```
// App Service
resource "azurerm_app_service" "app_service" {…
}

// App Service Plan
resource "azurerm_app_service_plan" "app_service_plan" {…
}

// Alert Rule
resource "azurerm_monitor_metric_alert" "alert_rule" {
  name                = "copilot-alert-rule"
  resource_group_name = azurerm_resource_group.rg.name
  scopes              = [azurerm_app_service.app_service.id]
  description         = "Alert rule for copilot app"
  severity            = 3
  window_size         = "PT5M"
  frequency           = "PT1M"
  criteria {
    metric_namespace = "Microsoft.Web/sites"
    metric_name      = "Requests"
    aggregation      = "Total"
    operator         = "GreaterThan"
    threshold        = 10
    dimension {
      name     = "ResultType"
      operator = "Include"
      values   = ["Success"]
    }
  }
  action {
    action_group_id = azurerm_monitor_action_group.action_group.id
  }
}
```

Alerts and Monitoring

KQL query prompts:

```
// Prompt: "Write a KQL query for Azure Log Analytics that:
// - Finds all HTTP 5xx errors from App Service logs in the last 4 hours
// - Groups by endpoint and exception type
// - Orders by frequency descending“
```

Azure Monitor alert rule (Bicep):

```
// Prompt: "Generate a Bicep module for an Azure Monitor metric alert that:
// - Triggers when App Service response time > 2 seconds for 5 consecutive minutes
// - Sends email to mercury-ops@contoso.com and posts to Teams channel via action
group
// - Has auto-mitigate set to true“
```

Dashboard prompt:

```
"Generate a KQL workbook JSON for an Azure Dashboard showing:
- Request rate, failure rate, and response time for mercury-api
- Separate tabs for live (last 1h) and historical (last 7d) views"
```

Log Analysis

Help investigate and identify issues in log files

whats the reason for exceptions in this file

GitHub Copilot

Used 1 reference >

From the provided log excerpt, there are two types of exceptions:

1. `Database connection timeout on query`: This error typically occurs when a database query takes longer than the configured timeout duration. This could be due to network issues, database server performance, or the complexity of the query itself.

2. `NullReferenceException in CartController.AddToCart`: This error occurs when you try to access a member on a type whose value is `null`. In this context, it's happening in the `AddToCart` method of the `CartController`. This could be due to an uninitialized object, a missing value that the method expects, or a failed method call that returns `null`.

Please note that these are just the error messages. To find the exact cause of these exceptions, you would need to look at the stack trace (if available) or debug the application.

```
3  2024-01-31 09:03:00 INFO:: HTTP GET Request for '/home/index'.
4  2024-01-31 09:04:00 WARN:: Low memory warning.
5  2024-01-31 09:05:00 ERROR:: Database connection timeout on query.
6  2024-01-31 09:06:00 INFO:: HTTP POST Request for '/account/login' with status 200 OK.
7  2024-01-31 09:07:00 DEBUG:: User authentication successful for user 'admin'.
8  2024-01-31 09:08:00 ERROR:: NullReferenceException in CartController.AddToCart.
9  2024-01-31 09:09:00 INFO:: Scheduled task 'UpdateProductCatalog' started.
10 2024-01-31 09:10:00 INFO:: Scheduled task 'UpdateProductCatalog' completed successful
11 2024-01-31 09:11:00 INFO:: Application start.
12 2024-01-31 09:12:00 DEBUG:: Initializing database connections.
13 2024-01-31 09:13:00 INFO:: HTTP GET Request for '/home/index'.
14 2024-01-31 09:14:00 WARN:: Low memory warning.
15 2024-01-31 09:15:00 ERROR:: Database connection timeout on query.
16 2024-01-31 09:16:00 INFO:: HTTP POST Request for '/account/login' with status 200 OK.
17 2024-01-31 09:17:00 DEBUG:: User authentication successful for user 'admin'.
18 2024-01-31 09:18:00 ERROR:: NullReferenceException in CartController.AddToCart.
19 2024-01-31 09:19:00 INFO:: Scheduled task 'UpdateProductCatalog' started.
20 2024-01-31 09:20:00 INFO:: Scheduled task 'UpdateProductCatalog' completed successful
21 2024-01-31 09:21:00 INFO:: Application start.
22 2024-01-31 09:22:00 DEBUG:: Initializing database connections.
23 2024-01-31 09:23:00 INFO:: HTTP GET Request for '/home/index'.
24 2024-01-31 09:24:00 WARN:: Low memory warning.
25 2024-01-31 09:25:00 ERROR:: Database connection timeout on query.
26 2024-01-31 09:26:00 INFO:: HTTP POST Request for '/account/login' with status 200 OK.
27 2024-01-31 09:27:00 DEBUG:: User authentication successful for user 'admin'.
28 2024-01-31 09:28:00 ERROR:: NullReferenceException in CartController.AddToCart.
```

Log Analysis

Structured investigation workflow:

Step 1 — Paste raw logs into Copilot Chat:

```
@copilot Here are the last 50 lines from our App Service logs during the outage
window.
Identify the root cause and timeline of failures:
[paste log content]
```

Step 2 — Generate a KQL query to replicate the pattern:

```
Based on the error pattern you identified, write a KQL query I can save as an Azure
Monitor Saved Search
to detect this condition proactively in future.
```

Step 3 — Generate a runbook:

```
Write a markdown runbook for the on-call engineer describing how to diagnose and
resolve this type of error.
Include the exact KQL queries, Azure CLI commands to check, and escalation path.
```

Helping you become a shell superhero…

Copilot CLI — Your Command Line Co-Pilot

Mode	How to start	Best for
Interactive	Type copilot	Iterative tasks, debugging pipelines, multi-step infra work
Programmatic	copilot -p "your prompt"	Scripted automation, CI/CD integration

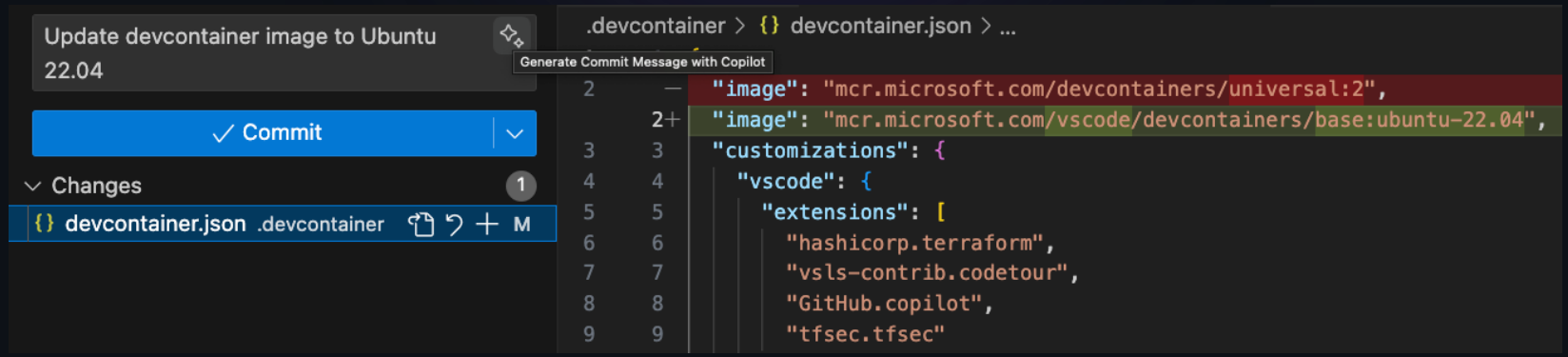
```
C:\cmder
λ
```

Copilot CLI — Your Command Line Co-Pilot

```
Programmatic use in a pipeline (headless)
copilot -p "Summarise all commits on this branch since last release tag" \
  --allow-tool='shell(git)'


# Allow only specific safe tools (recommended for CI use)
copilot -p "Run terraform fmt on all .tf files" --allow-tool='shell(terraform)'

# Deny destructive commands while allowing everything else
copilot --allow-all-tools --deny-tool='shell(rm)' --deny-tool='shell(git push)'
```

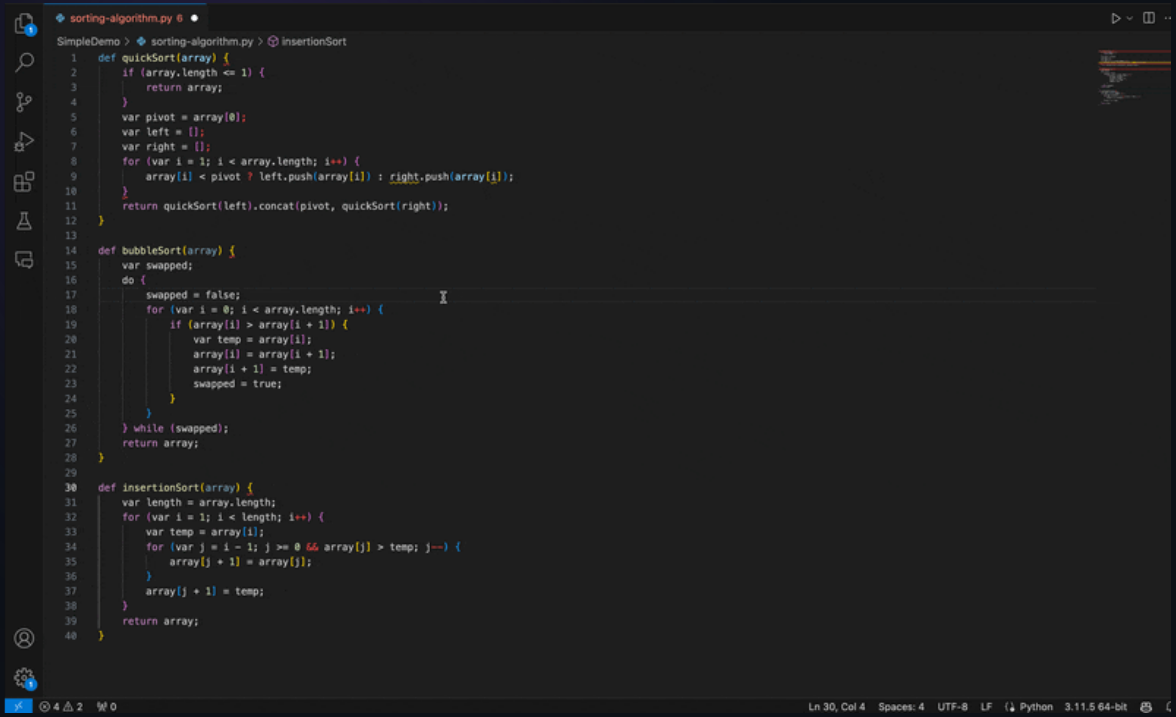
Commit messages

Automatically generate your commit messages



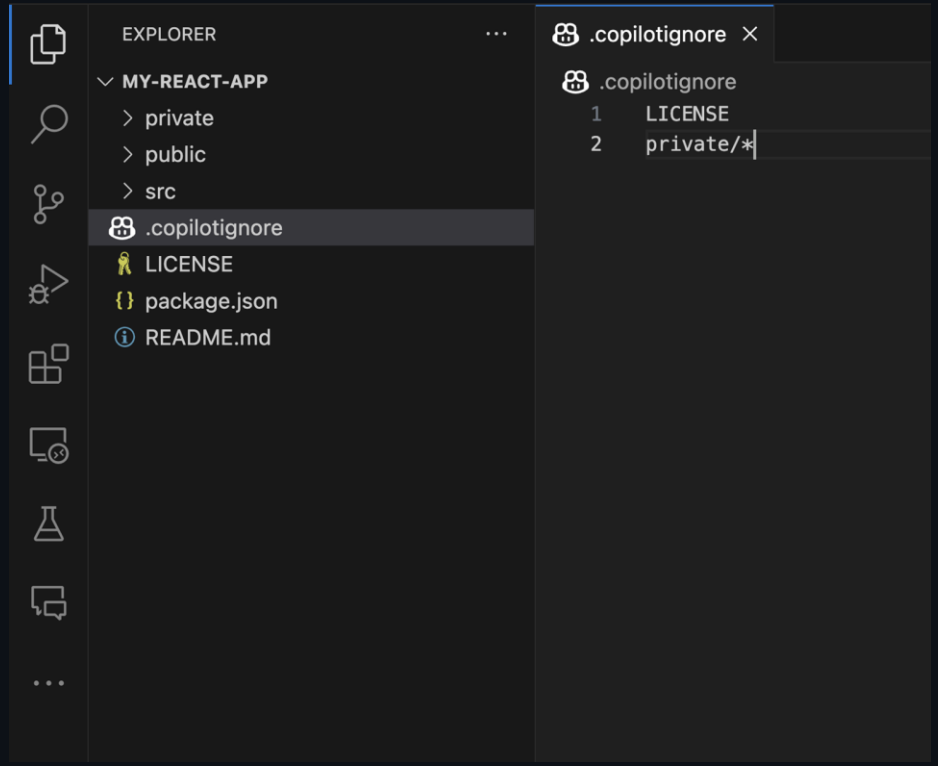
In-file Copilot options

Copilot now offers **in-file Copilot feature** to selectively improve



Block files from Copilot

Use .copilotignore to block files and folders from being used by Github Copilot



Block files from Copilot

Ignore specific **repositories and paths** in Restricted Content

evil-copilot
Organization, part of Evil-Copilot Switch to another account

Go to your organization profile

General
Features

Access
Billing and plans
Repository roles
Member privileges
Import/Export
Moderation

Code, planning, and automation
Repository
Codespaces
Copilot
Access
Policies and features
Restricted content Beta
Actions
Webhooks
Discussions
Packages
Pages
Projects

Security
Authentication security

Restricted content

Beta Give feedback

Restricted paths

Choose the repositories and paths within your organization that GitHub Copilot should ignore. **Copilot won't be able to access or utilize the contents** located in those specified paths. Copilot restrictions are currently supported in Visual Studio Code. Learn more.

Use YAML syntax with fnmatch notation to write rules and target specific paths (folders and files) as needed. GitHub Copilot will ignore paths that conform to the provided patterns.

Repositories and paths to ignore:

```
1   "*": ["**/.env"]
2
3   marais-testing-repo:
4     - "/src/kernel.rs"
5
6   git@github.com:primer/react.git:
7     - "e2e/*"
8
9   git@gitlab.com:opencity-labs/surreal-react.git:
10    - "fly.toml"
11    - "**/surrealdbjs/*"
```

Use `Control + Shift + m` to toggle the `tab` key moving focus. Alternatively, use `esc` then `tab` to move to the next interactive element on the page.

Use separate lines to choose what folders and files will by ignored by GitHub Copilot

maraisr last edited 1 minute ago

Discard changes Save changes

Set the scene for GitHub Copilot

Set the context for suggestions using #file, #select, or @workspace

CHAT

@workspace /explain #file:BankAccount.cs How can I implement authentication in the selected code

GitHub Copilot used @workspace /explain

Used 2 references

The selected code already has a basic form of authentication implemented. Here's how it works:

1. The `BankAccount` class has a private field `password` that is set when a new `BankAccount` object is created. This is done in the constructor method `BankAccount`.

```
private string password;

public BankAccount(string accountNumb
{
    // Existing code...

    this.password = password;
}
```

2. The `Authenticate` method checks if the provided password matches the password stored in the `password` field.

```
public bool Authenticate(string passw
{
    return this.password == password;
}
```

BankAccount.cs

C: > Temp > BankAccount.cs > BankAccount

```
1 reference
public class BankAccount
{
    // Existing code...

    2 references
    private string password;

    0 references
    public BankAccount(string accountNumber,
    {
        // Existing code...

        this.password = password;
    }

    1 reference
    public bool Authenticate(string password)
    {
        return this.password == password;
    }

    0 references
    public void Credit(double amount, string
    {
        if (!Authenticate(password))
        {
            throw new Exception("Authenticati
        }

        // Existing code...
    }

    // Similar changes would be made to the D
}
```

Terminal Commands

@terminal to suggest terminal commands from Copilot chat.

@terminal deploy a node js application to an azurer app service

GitHub Copilot used @terminal

Used 15 references

To deploy a Node.js application to Azure App Service, you can use the Azure CLI. Here's a step-by-step command:

1. Login to Azure:

```
az login
```

2. Create a resource group (if it doesn't exist):

```
az group create --name {resource_group_name} -
```

3. Create an App Service plan:

```
az appservice plan create --name {plan_name} -
```

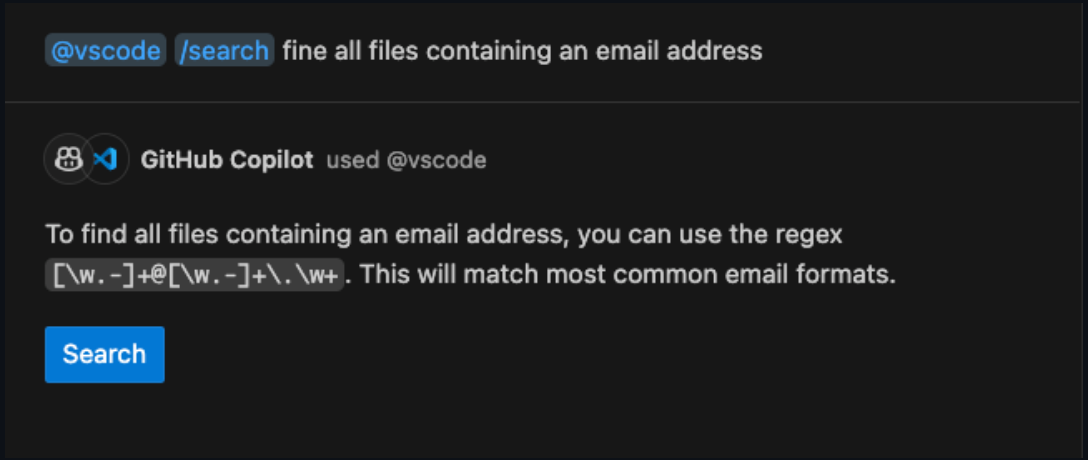
4. Create a web app:

```
az webapp create --resource-group {resource_gr
```

5. Deploy your Node.js app:

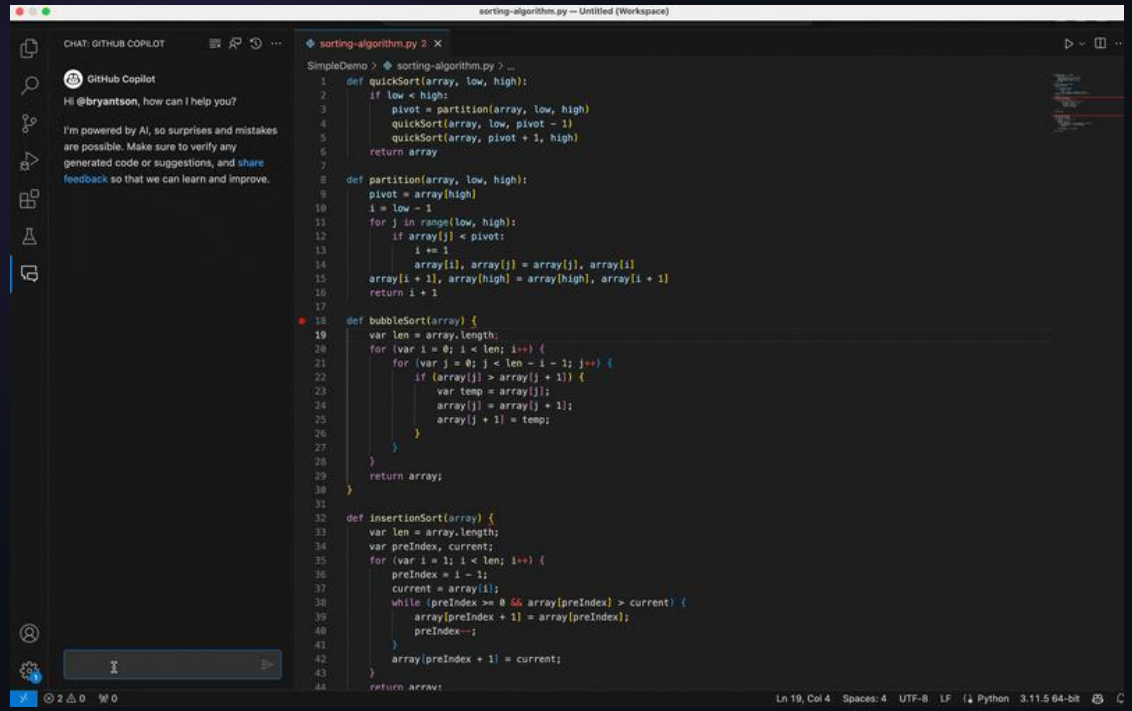
Search

@vscode /search
to generate search terms and patterns for Visual Studio Code



/fix, /tests and other / command options

Leverage / command options that can help to improve your code



Convert code between languages

Convert code from one programming language to another

CHAT

convert this from azure pipeline to a github actions workflow assuming reusable workflows are in place where azure pipeline templates are used

GitHub Copilot

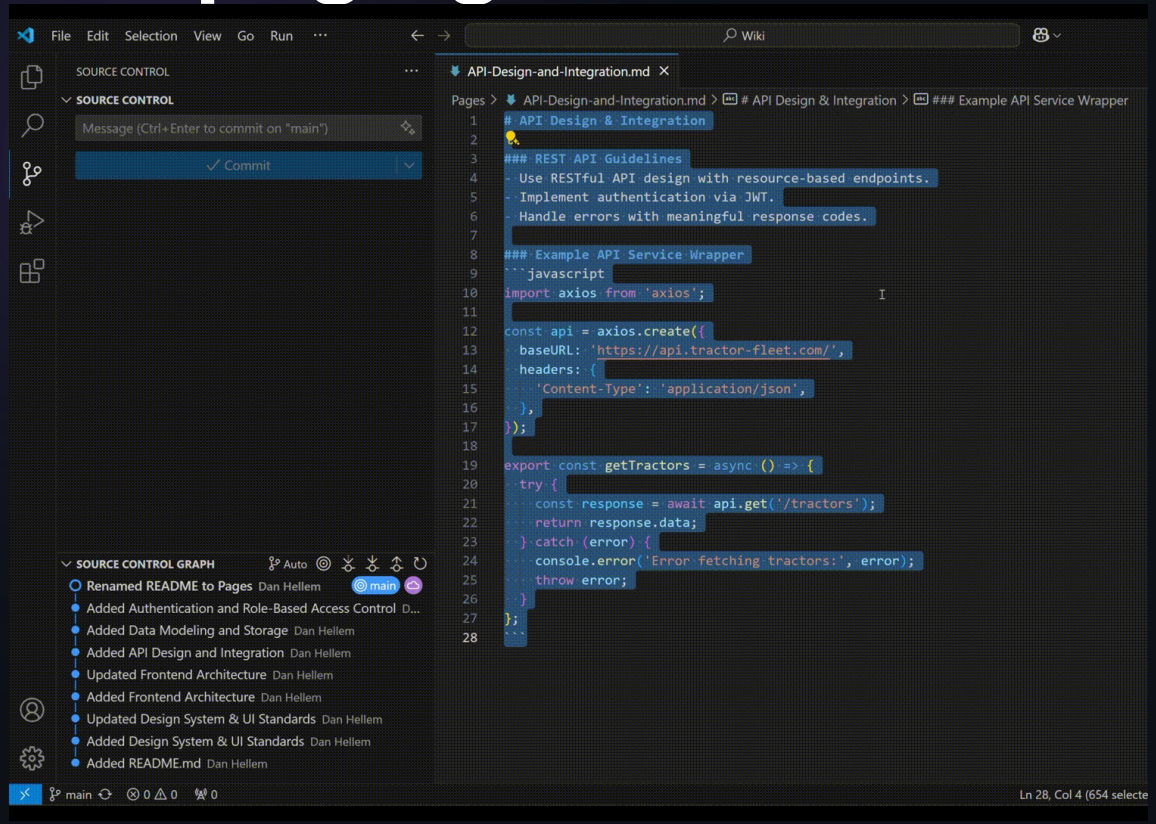
Used 1 reference

Here's a conversion of your Azure Pipelines YAML to a GitHub Actions workflow. Note that GitHub Actions doesn't have a direct equivalent for Azure Pipelines templates, but you can use reusable workflows in GitHub Actions.

```yaml
name: CI

on:
  push:
    branches:
      - main
    paths:
      - 'bicep/policy'

env:
  templateFolderPath: ${{ github.work
  templateFileName: policy.bicep
  parameterFilePath: ${{ github.works
  location: "[ENTER LOCATION]"
  subscriptionId: ""
  svcConnection: "[ENTER SERVICE CONN
  adoEnvironment: "policy"
  deploymentName: "deploy_policy"
  managementGroupId: "[COMPANY ROOT M

jobs:
  build:
    uses: ./.github/workflows/build-t
    with:
```

Wiki markdown page generation

Making it easier to document best practices, guidelines, and technical references.



MCP Server Integration

- AWS KB Retrieval - Retrieval from AWS Knowledge Base using Bedrock Agent Runtime
- Brave Search - Web and local search using Brave's Search API
- EverArt - AI image generation using various models
- Everything - Reference / test server with prompts, resources, and tools
- Fetch - Web content fetching and conversion for efficient LLM usage
- Filesystem - Secure file operations with configurable access controls
- Git - Tools to read, search, and manipulate Git repositories
- GitHub - Repository management, file operations, and GitHub API integration
- GitLab - GitLab API, enabling project management
- Google Drive - File access and search capabilities for Google Drive
- Google Maps - Location services, directions, and place details
- Memory - Knowledge graph-based persistent memory system
- PostgreSQL - Read-only database access with schema inspection
- Puppeteer - Browser automation and web scraping
- Redis - Interact with Redis key-value stores
- Sentry - Retrieving and analyzing issues from Sentry.io
- Sequential Thinking - Dynamic and reflective problem-solving through thought sequences
- Slack - Channel management and messaging capabilities
- Sqlite - Database interaction and business intelligence capabilities
- Time - Time and timezone conversion capabilities

CHAT

List all the app services in my sub c1a6ecdc-2290-4a0c-b941-f065b2c68131

I'll list all the App Services in your subscription.

> Learning about appservice capabilities... – Azure MCP (MCP Server) ✓

> Learning about appservice capabilities... – Azure MCP (MCP Server) ✓

Let me use the Azure resource query tool to list the App Services:

> Using "@azure query azure resource graph" ✓

There is **1 App Service** in subscription `c1a6ecdc-2290-4a0c-b941-f065b2c68131`:

- **arinco-crew-nearby** (microsoft.web/sites) - Location: Australia Southeast

Find unused resources and provide script to remove them

I'll help you find unused Azure resources in your subscription and provide a script to remove them.

> Using "@azure query azure resource graph" ✓

I can see you have **201 resources** in the subscription. Let me analyze different resource types to identify potentially unused resources:

> Using "@azure query azure resource graph" ✓

> Using "@azure query azure resource graph" ✓

> Using "@azure query azure resource graph" ✓

> Using "@azure query azure resource graph" ✓

> Using "@azure query azure resource graph" ✓

> Using "@azure query azure resource graph" ✓

Perfect! I found several unused resources. Let me create a comprehensive PowerShell script to remove them:

✓ Created Remove-UnusedAzureResources.ps1

Unused Resources Found

I've identified **7 unused resources** in your subscription and created a cleanup script:

Deallocated Virtual Machines (3)

- **mcndc01** - mcn-ad-recovery-test-rg (deallocated since July 2024)
- **mcndc02** - mcn-ad-recovery-test-rg (deallocated since July 2024)
- **mcnwork01** - mcn-ad-recovery-test-rg (deallocated since July 2024)

Unattached Network Interfaces (3)

- **dl-dbricks-ui-pe-nic** - dl-dev-rg
- **kube-apiserver.nic.24563277-0a9a-4183-8b02-9c5e601f6f9b** - hb-ae-test-truckapi-nodes-rg
- **tor-ae-ai-aif-account-pe-nic** - tor-ae-ai-rgp

> 1 file changed +201 -1 Keep Undo

mcp.json

Describe what to build next

Agent Claude Sonnet 4.5

All MCP servers 55

Markitdown Install
Convert various file formats (PDF, Word, Excel, images, audio) to Markdown.
By microsoft 84,142

Netdata Install
AI-powered infrastructure monitoring with real-time metrics, logs, alerts, and ML anomaly detection.
By netdata 76,975

Context7 Install
Up-to-date code docs for any prompt
By upstash 39,120

GitHub Install
Connect AI assistants to GitHub - manage repos, issues, PRs, and workflows through natural language.
By github 25,099

Playwright Install
Automate web browsers using accessibility trees for testing and data extraction.
By microsoft 24,241

Serena Install
Semantic code retrieval & editing tools for coding agents.
By oraios 17,057

ChromeDevTools Install
MCP server for Chrome DevTools
By ChromeDevTools 16,350

Firecrawl Install
Extract web data with Firecrawl
By firecrawl 5,051

Unity Install
Control the Unity Editor from MCP clients via a Unity bridge + local Python server.
By CoplayDev 4,227

Notion Install
Official MCP server for Notion API
By makenotion 3,573

Supabase Install
MCP server for interacting with the Supabase platform
By supabase-community 2,318

Azure MCP Server Install
All Azure MCP tools to create a seamless connection between AI agents and Azure services.
By microsoft 2,285

Stripe Install

Microsoft Learn Install

Terraform Install

Mercury DevOps Agent Skill

```
---
name: mercury-devops
description: >
  Auto-load Mercury DevOps standards: Azure naming conventions, Terraform module patterns,
  Azure DevOps pipeline templates, PowerShell deployment scripts, and KQL query standards.
  Use when working on infrastructure, pipelines, scripts, monitoring, or deployments.
---

## Mercury DevOps Standards

### Naming Convention
- Resource groups: `rg-{app}-{env}-{region}` (e.g., `rg-mercury-prod-we`)
- App Services: `app-{name}-{env}-{region}`
- Key Vaults: `kv-{app}-{env}` (max 24 chars)
- Storage Accounts: `st{app}{env}{region}` (lowercase, no hyphens, max 24 chars)

### Terraform Standards
- Provider: `azurerm ~> 3.0`
- Always include `tags` block: CostCentre, Team, Environment, ManagedBy
- Use `terraform.tfvars` for environment values, never hardcode

### Pipeline Standards
- .NET builds must include: restore → build → test (with coverage) → publish artifacts
- Coverage minimum: 80% line coverage
- Always use variable groups for secrets, never pipeline variables

### PowerShell Standards
- Minimum version: PowerShell 7
- Always include `-ErrorAction Stop` on Azure CLI calls
- Log to Azure Log Analytics using `Send-AzOperationalInsightsData`
```

Common Problems

Problems #1:

Copilot generates invalid infrastructure

Why it happens:

Azure resource naming rules and character limits are highly specific
Copilot may generate valid-looking Terraform that fails `terraform validate`
`azurerm` provider APIs change frequently; training data may lag

Solutions:

Always run terraform validate and terraform plan before applying
Use `#file:main.tf` to give Copilot your existing working modules as reference
Prompt: "Validate this Terraform against Azure naming restrictions and `azurerm ~> 3.0 provider`"

Problems #2:

Generated pipelines don't match your organisation's templates

Why it happens:

Copilot generates generic pipeline YAML, unaware of your required `extends` templates
Mandatory security scanning steps (Checkmarx, Mend) are omitted
Agent pool names, service connection names are guessed

Solutions:

Reference existing pipeline files: `#file:azure-pipelines.yml`
Prompt prefix: "Generate pipeline YAML that extends our template at `.pipelines/templates/dotnet-base.yml`"
Use the `mercury-devops` agent skill to pre-load pipeline standards automatically

Problem #3:

Secrets Exposure Risk

Why it happens:

Copilot optimises for working examples; secrets in examples are common in training data
Quick demo snippets often hardcode connection strings

Solutions:

Always review generated scripts for hardcoded strings before committing
Add `.github/copilot-instructions.md` rule: "Never hardcode credentials; always use Azure Key Vault references or environment variables"
Use `.copilotignore` to exclude .env and `terraform.tfvars` from Copilot context

Q & A

Hands-on Exercise

- Terraform
- https://github.com/shin-akuma/copilot-terraform
- Scripting
- https://experience.cloudlabs.ai/#/labguidepreview/f9fd80ac-fc1a-4609-a3f1-06650aec389e (Page 10)
- https://github.com/shin-akuma/copilot-scripting

Hands-on Exercise

- Open the https://github.com/MicrosoftLearning/mslearn-github-copilot-dev/raw/refs/heads/main/DownloadableCodeProjects/Downloads/AZ2007LabAppM4.zip
- Ask Copilot to generate an Azure DevOps pipeline for the application build/publish
- Add the mercury-devops skill and regenerate — compare the two outputs

Hands-on Exercise

- Install Copilot CLI: winget install GitHub.CopilotCLI (or npm install -g @github/copilot)
- Run copilot interactively and complete 3 Azure CLI tasks using natural language — no documentation allowed
- Try /plan mode (Shift+Tab) on a non-trivial task to see the structured planning workflow

Upcoming Sessions

1 Q&A Session

Thank you